# Clase 9 · hostent, conversión de IP, puertos y ciclo de vida del socket

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 24 sep 2026 · Tema 2 · Apuntes y resumen de estudio

## PARTE 1 · Apuntes completos

### 1. gethostbyname() y struct hostent

`gethostbyname(const char *name)` consulta hosts o DNS y devuelve un apuntador a `struct hostent` (o **NULL** si falla).

| Campo | Tipo | Contenido |
|---|---|---|
| `h_name` | `char *` | Nombre oficial del host |
| `h_aliases` | `char **` | Alias, terminado en NULL |
| `h_addrtype` | entero | Tipo de dirección (AF_INET) |
| `h_length` | entero | Bytes de la dirección (4 en IPv4) |
| `h_addr_list` | `char **` | Lista de IP del dominio |

**Cast obligatorio**

`h_addr_list` está declarado como `char` **, pero las IP son `struct in_addr`: hay que hacer `struct in_addr` `addr_list = (struct in_addr )` `he->h_addr_list;` y recorrer hasta el** NULL**:

```
for (int i = 0; addr_list[i] != NULL; i++)
    printf("IP %d: %s\n", i + 1, inet_ntoa(*addr_list[i]));
```

Se imprime `i + 1` para que el usuario vea "IP 1".

### 2. inet_ntoa e inet_aton

Nemotecnia: **inet** = Internet, **n** = network (binario de red), **a** = ASCII (decimal punteado), **to** = hacia.

| Función | Convierte |
|---|---|
| `inet_ntoa()` | `struct in_addr` binaria → cadena "142.251.x.x" |
| `inet_aton()` | Cadena → `struct in_addr` |

Al pasar `addr_list[i]` (apuntador) a `inet_ntoa()` hay que **desreferenciar** con `*`.

## 3. Demo: dominios y alta disponibilidad

| Dominio | Resultado |
|---|---|
| www.google.com | **8 IP** (142.251.x.x, Mountain View); mismas IP en distinto orden para cada alumno |
| google.com | 1 IP (el `www` apunta a la granja web) |
| www.unam.mx | 2 IP (104.17.118.46 y .117.46): Cloudflare, alta disponibilidad de 2 nodos |
| www.ingenieria.unam.mx | 1 IP (132.248.54.20): servidor local de la UNAM |

**Clúster de alta disponibilidad:** dos o más servidores coordinados que se comportan como uno.

| Estrategia | Cómo reparte | Cuándo |
|---|---|---|
| Por capacidad / jerarquía | Todo al principal; al acercarse al 100 % pasa al segundo, etc. | Equipos de distinta capacidad |
| **Round-robin** | Cíclico: 1 → servidor 1, …, 8 → 8, 9 → 1 | Hardware similar |

## 4. Programa 3: cuatro formas de extraer la IP

1. `inet_ntoa()`. 2. **`inet_ntop()`** (moderna y segura, IPv4 e IPv6). 3. Por **octetos**: cast de `h_addr` a `(unsigned char *)` e imprimir los 4 bytes. 4. Valor crudo de 32 bits. También recorre `h_aliases`.

## 5. Puertos lógicos (IANA)

Número de **16 bits sin signo → 65,536 puertos (0–65535)**, asignados por la **IANA**.

| Rango | Nombre | Regla |
|---|---|---|
| **0–1023** | Del sistema (reservados) | Solo root/sudo |
| **1024–49151** | De usuario (registrados) | Cualquier usuario si está libre |
| **49152–65535** | Dinámicos o privados | Conexiones efímeras |

Emblemáticos: 21 FTP, 22 SSH, 23 Telnet (texto plano), 25 SMTP, 80 HTTP (Tim Berners-Lee), 443 HTTPS, 666 Doom.

## 6. Ciclo de vida de un socket

| Servidor | Cliente |
|---|---|
| `socket()` → inicializar `sockaddr_in` (AF_INET, IP, puerto) | `socket()` → `sockaddr_in` con IP y puerto del servidor |
| `bind()` (liga IP y puerto) → `listen()` (cola/backlog) | — |
| `accept()` (bloquea; devuelve un descriptor nuevo por cliente) | `connect()` (al nivel de `accept()`) |
| Intercambio: `send()/recv()` o `write()/read()` | Ídem |

| | |
|---|---|
| `close()` del cliente y regresa a `accept()` | `close()` y termina |

**Regla de emparejamiento**

Si un extremo usa `send()`, el otro usa `recv()`; si uno usa `write()`, el otro `read()`. **Nunca mezclar.**

- **Síncrona:** alternancia estricta (tras un `send()` no se envía otro hasta el `recv()`).
  **Asíncrona:** n mensajes seguidos.

## PARTE 2 · Resumen para estudiar

*Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.*

**DNS en C.** `gethostbyname()` → `struct hostent`; cast de `h_addr_list` a `(struct in_addr **)` y recorrer hasta NULL; validar NULL.

> **Conversión de IP (examen)**
>
> `inet_ntoa`: network (binario) → ASCII. `inet_aton`: ASCII → network. `inet_ntop`: moderna (IPv4/IPv6).

| Tema | Para recordar |
|---|---|
| Puertos IANA | 0–1023 root; 1024–49151 usuario; 49152–65535 dinámicos |
| Alta disponibilidad | Varias IP por dominio; jerarquía o round-robin |
| Ciclo del servidor | socket → sockaddr_in → bind → listen → accept → datos → close |
| Ciclo del cliente | socket → sockaddr_in → connect → datos → close |
| Parejas | send/recv, write/read — nunca mezclar |